# Unit 5

# Web Security Considerations

- The World Wide Web is fundamentally a client/server application running over the Internet and TCP/IP intranets
- The following characteristics of Web usage suggest the need for tailored security tools:
  - Web servers are relatively easy to configure and manage
  - Web content is increasingly easy to develop
  - The underlying software is extraordinarily complex
    - May hide many potential security flaws
  - A Web server can be exploited as a launching pad into the corporation's or agency's entire computer complex
  - Casual and untrained (in security matters) users are common clients for Web-based services
    - Such users are not necessarily aware of the security risks that exist and do not have the tools or knowledge to take effective countermeasures

# Web Security Threat Classification

## Passive and Active threats

Passive attacks include eavesdropping on network traffic between browser and server and gaining access to information on a Web site that is supposed to be restricted. Active attacks include impersonating another user, altering messages in transit between client and server, and altering information on a Web site.

### Attacks

- Passive attacks
  - Interception
    - Release of message contents
    - Traffic analysis
- Active attacks
  - Interruption, modification, fabrication
    - Masquerade
    - Replay
    - Modification
    - Denial of service

## Location of threat

- Web Browser
- Web Server
- Network Traffic

| | Threats | Consequences | Countermeasures |
|---|---|---|---|
| **Integrity** | •Modification of user data<br>•Trojan horse browser<br>•Modification of memory<br>•Modification of message traffic in transit | •Loss of information<br>•Compromise of machine<br>•Vulnerabilty to all other threats | Cryptographic checksums |
| **Confidentiality** | •Eavesdropping on the net<br>•Theft of info from server<br>•Theft of data from client<br>•Info about network configuration<br>•Info about which client talks to server | •Loss of information<br>•Loss of privacy | Encryption, Web proxies |
| **Denial of Service** | •Killing of user threads<br>•Flooding machine with bogus requests<br>•Filling up disk or memory<br>•Isolating machine by DNS attacks | •Disruptive<br>•Annoying<br>•Prevent user from getting work done | Difficult to prevent |
| **Authentication** | •Impersonation of legitimate users<br>•Data forgery | •Misrepresentation of user<br>•Belief that false information is valid | Cryptographic techniques |

Table 17.1 A Comparison of Threats on the Web

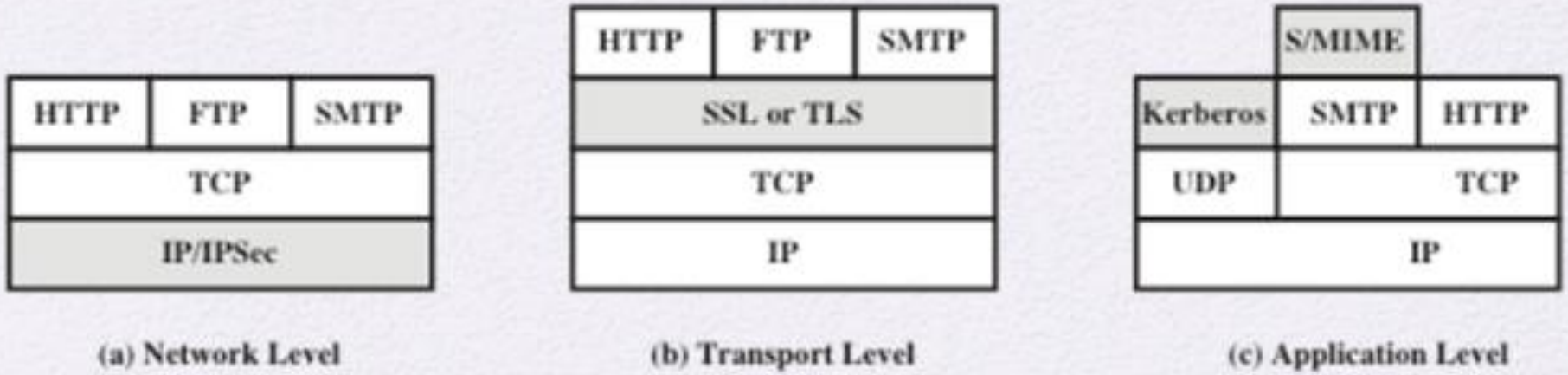



**Figure 17.1 Relative Location of Security Facilities in the TCP/IP Protocol Stack**

**Transport Layer Security (TSL)**; the current version is Version 1.2, defined in RFC 5246. TLS is an Internet standard that evolved from a commercial protocol known as **Secure Sockets Layer (SSL)**.

Although SSL implementations are still around, it has been deprecated by IETF and is disabled by most corporations offering TLS software.

# Secure Sockets Layer (SSL)

- One of the most widely used security services
- A general purpose service implemented as a set of protocols that rely on TCP
  - Could be provided as part of the underlying protocol suite and therefore be transparent to applications
  - Can be embedded in specific packages

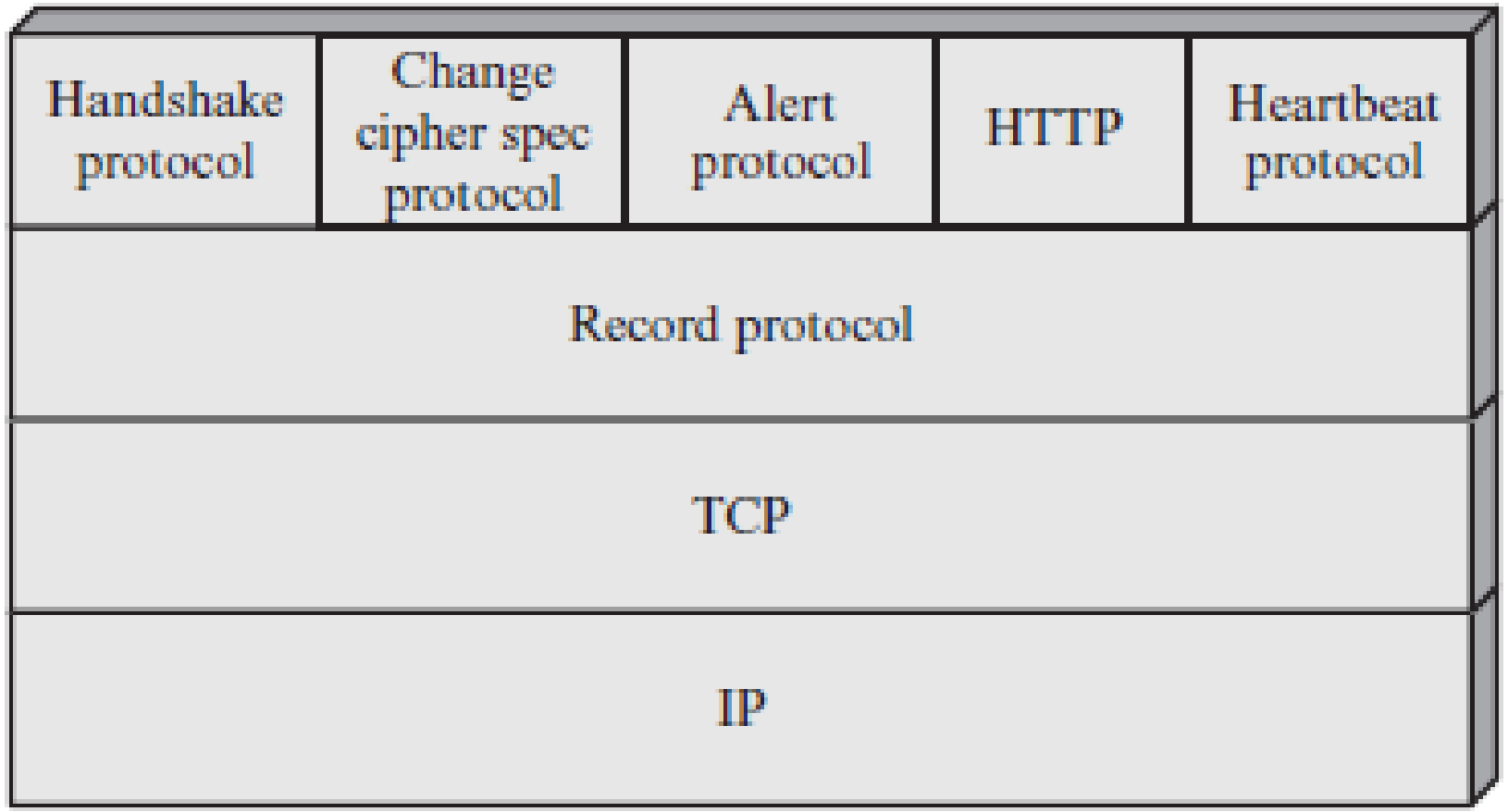



Figure 17.2 TLS Protocol Stack

# TLS Handshake – Pre-Requisites

- SSL/TLS have three goals:
  - **Confidentiality** – Provided by Symmetric Encryption
  - **Integrity** – Provided by M.A.C. (Hashing)
  - **Authentication** – Provided by Certificates / PKI

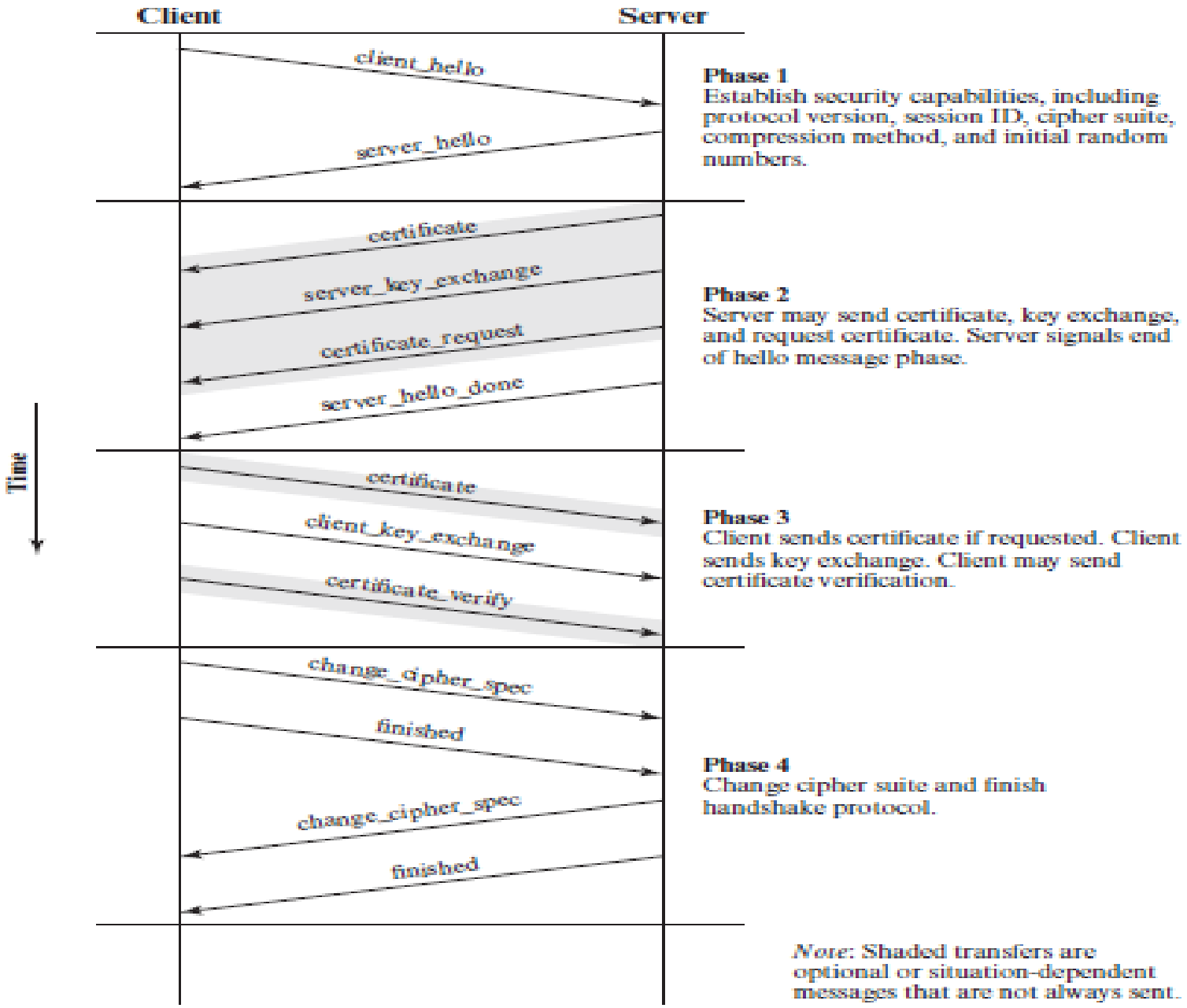



Figure 17.6 Handshake Protocol Action

# Phase 1 – Handshake Protocol

| 1 byte | 3 bytes | ≥ 0 bytes |
|---|---|---|
| Type | Length | Content |

**(c) Handshake Protocol**

- Type: Indicates one of 10 messages. Table 17.2 lists the defined message

Table 17.2 TLS Handshake Protocol Message Types

| Message Type | Parameters |
|---|---|
| hello_request | null |
| client_hello | version, random, session id, cipher suite, compression method |
| server_hello | version, random, session id, cipher suite, compression method |
| certificate | chain of X.509v3 certificates |
| server_key_exchange | parameters, signature |
| certificate_request | type, authorities |
| server_done | null |
| certificate_verify | signature |
| client_key_exchange | parameters, signature |
| finished | hash value |

- **Length (3 bytes):** The length of the message in bytes.
- ■ **Content (# 0 bytes):** The parameters associated with this message; these are listed in Table

# Client Hello

- **Version:** The highest TLS version understood by the client.
- **Random:** A client-generated random structure consisting of a 32-bit timestamp and 28 bytes generated by a secure random number generator. These values serve as nonces and are used during key exchange to prevent replay attacks.
- **Session ID:** A variable-length session identifier. A nonzero value indicates that the client wishes to update the parameters of an existing connection or to create a new connection on this session. A zero value indicates that the client wishes to establish a new connection on a new session.
- **CipherSuite:** This is a list that contains the combinations of cryptographic algorithms supported by the client, in decreasing order of preference. Each element of the list (each cipher suite) defines both a key exchange algorithm and a CipherSpec; these are discussed subsequently.
- **Compression Method:** This is a list of the compression methods the client supports.

# Server Hello

- Version field contains the lowest of the version suggested by the client and the highest supported by the server.
- The Random field is generated by the server and is independent of the client's Random field.
- If the SessionID field of the client was nonzero, the same value is used by the server; otherwise the server's SessionID field contains the value for a new session.
- The CipherSuite field contains the single cipher suite selected by the server from those proposed by the client.
- The Compression field contains the compression method selected by the server from those proposed by the client.

## Key exchange methods supported

- RSA
- Fixed Diffie Hellman - This is a Diffie–Hellman key exchange in which the server's certificate contains the Diffie–Hellman public parameters signed by the certificate authority (CA).
- Ephemeral Diffie Hellman - This technique is used to create ephemeral (temporary, one-time) secret keys
- Anonymous Diffie Hellman - The base Diffie–Hellman algorithm is used with no authentication.

# Cipher Spec contains following information

- **CipherAlgorithm -** Any of the algorithms mentioned earlier: RC4, RC2, DES, 3DES, DES40, or IDEA
- **MACAlgorithm:** MD5 or SHA-1
- **CipherType:** Stream or Block
- **IsExportable:** True or False
- **HashSize:** 0, 16 (for MD5), or 20 (for SHA-1) bytes
- **Key Material:** A sequence of bytes that contain data used in generating the write keys
- **IV Size:** The size of the Initialization Value for Cipher Block Chaining (CBC) encryption

# Phase 2 - *SERVER AUTHENTICATION AND KEY EXCHANGE*

- The server begins this phase by sending its certificate if it needs to be authenticated; the message contains one or a chain of X.509 certificates.
- The **certificate message** is required for any agreed-on key exchange method except anonymous Diffie–Hellman.
- Note that if fixed Diffie–Hellman is used, this certificate message functions as the server's key exchange message because it contains the server's public Diffie–Hellman parameters.
- Next, a **server_key_exchange message** may be sent if it is required. It is not required in two instances: (1) The server has sent a certificate with fixed Diffie– Hellman parameters; or (2) RSA key exchange is to be used.
- It is needed for 1. **Anonymous Diffie–Hellman: 2. Ephemeral Diffie–Hellman, 3. RSA key exchange (in which the server is using RSA but has a signature-only RSA key)**

# A nonanonymous server (server not using anonymous Diffie–Hellman) can request a certificate from the client.

The **certificate_request message** includes two parameters: certificate_type and certificate_authorities. The certificate type indicates the public-key algorithm and its use:

- RSA, signature only
- DSS, signature only
- RSA for fixed Diffie–Hellman; in this case the signature is used only for authentication, by sending a certificate signed with RSA
- DSS for fixed Diffie–Hellman; again, used only for authentication

- The second parameter in the certificate_request message is a list of the distinguished names of acceptable certificate authorities.
- The final message in phase 2, and one that is always required, is the **server_done message**, which is sent by the server to indicate the end of the server hello and associated messages. After sending this message, the server will wait for a client response. This message has no parameters.

# Phase 3: *CLIENT AUTHENTICATION AND KEY EXCHANGE*

- Upon receipt of the `server_done` message, the client should verify that the server provided a valid certificate (if required) and check that the `server_hello` parameters are acceptable. If all is satisfactory, the client sends one or more messages back to the server.
- If the server has requested a certificate, the client begins this phase by sending a **certificate message**. If no suitable certificate is available, the client sends a `no_certificate` alert instead.
- If the server has requested a certificate, the client begins this phase by sending a **certificate message**

  The content is

1. RSA, 2. Ephemeral or Anonymous Diffie–Hellman, 3. Fixed Diffie–Hellman

- Finally, in this phase, the client may send a **certificate_verify message** to provide explicit verification of a client certificate.

# Phase 4: Finish

- Phase 4 completes the setting up of a secure connection. The client sends a **change_cipher_spec message** and copies the pending CipherSpec into the current CipherSpec.
- Note that this message is not considered part of the Handshake Protocol but is sent using the Change Cipher Spec Protocol. The client then immediately sends the **finished message** under the new algorithms, keys, and secrets
- In response to these two messages, the server sends its own `change_cipher_spec` message, transfers the pending to the current CipherSpec, and sends its finished message. At this point, the handshake is complete and the client and server may begin to exchange application-layer data.

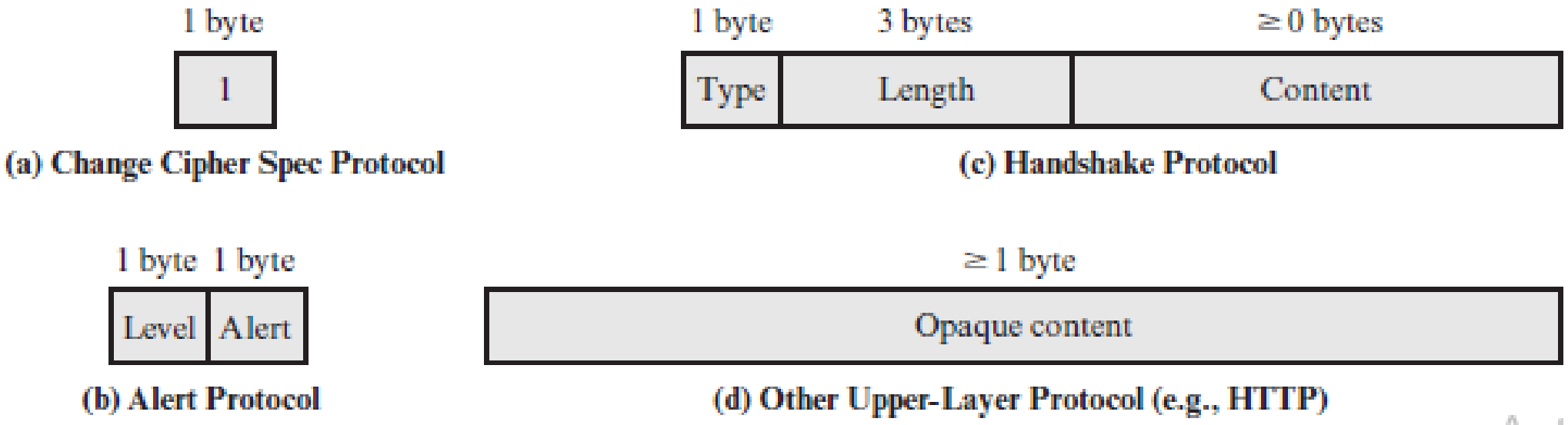



Figure 17.5 TLS Record Protocol Payload

Activa
Go to Se

Two important TLS concepts are the TLS session and the TLS connection, which are defined in the specification as follows:

- **Connection:** A connection is a transport (in the OSI layering model definition) that provides a suitable type of service. For TLS, such connections are peer-to-peer relationships. The connections are transient. Every connection is associated with one session.
- **Session:** A TLS session is an association between a client and a server. Sessions are created by the Handshake Protocol. Sessions define a set of cryptographic security parameters, which can be shared among multiple connections. Sessions are used to avoid the expensive negotiation of new security parameters for each connection.

## TLS Record Protocol

The TLS Record Protocol provides two services for TLS connections:

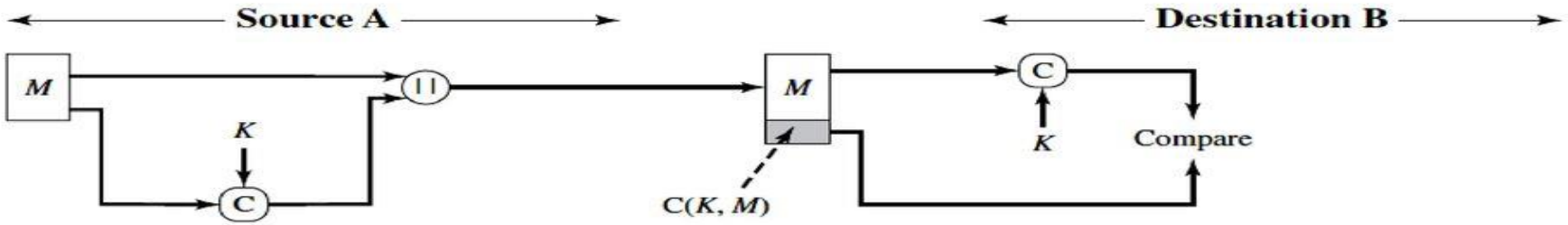
- **Confidentiality:** The Handshake Protocol defines a shared secret key that is used for conventional encryption of TLS payloads.
- **Message Integrity:** The Handshake Protocol also defines a shared secret key that is used to form a message authentication code (MAC).

Figure 17.2 indicates the overall operation of the TLS Record Protocol. The

# Message Authentication Code- for message integrity

## C. Message Authentication Code

- An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a **cryptographic checksum** or **MAC,** that is appended to the message.




(a) Message authentication

- A MAC function is similar to encryption. One difference is that the MAC algorithm need not be reversible, as it must be for decryption.
- In general, the MAC function is a many-to-one function.
- Because of the mathematical properties of the authentication function, it is less vulnerable to being broken than encryption.

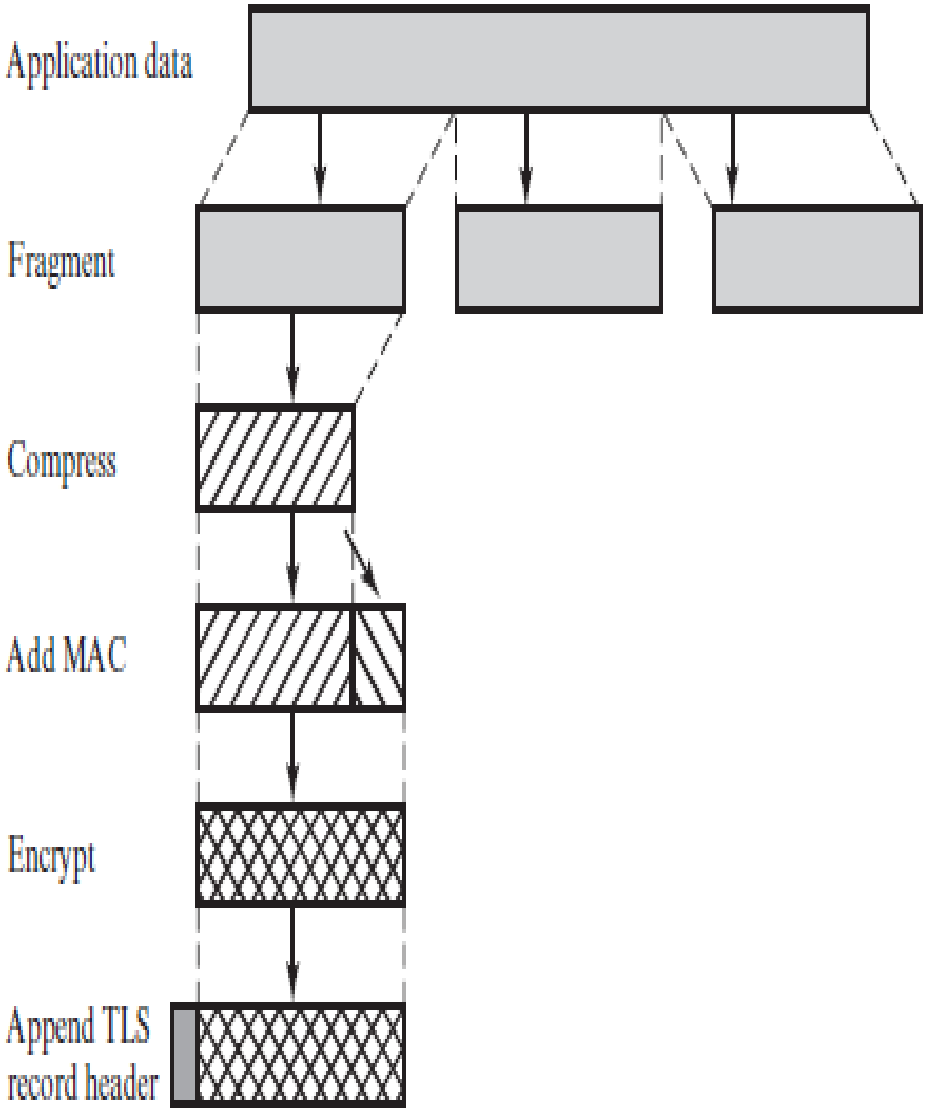



Figure 17.3 TLS Record Protocol Operation

- Step 1: Fragmentation blocks of 2 pow 14 bytes (16,384 bytes) or less.
- Step 2: Compression <1024 bytes. In TLSv2, no compression algorithm is specified, so the default compression algorithm is null.
- Step 3: Calculate and add MAC (next slide)
- Step 4: Do symmetric encryption

| Block Cipher | | Stream Cipher | |
|---|---|---|---|
| Algorithm | Key Size | Algorithm | Key Size |
| AES<br>3DES | 128, 256<br>168 | RC4-128 | 128 |

- Step 5: Add TLS record header

Content Type, Major Version, Minor Version and Compressed length

$$\mathrm{HMAC}_K(M) = \mathrm{H}[(K^+ \oplus \mathrm{opad}) \parallel \mathrm{H}[(K^+ \oplus \mathrm{ipad}) \parallel M]]$$

where

| | | |
|---|---|---|
| H | = | embedded hash function (for TLS, either MD5 or SHA-1) |
| $M$ | = | message input to HMAC |
| $K^+$ | = | secret key padded with zeros on the left so that the result is equal to the block length of the hash code (for MD5 and SHA-1, block length = 512 bits) |
| ipad | = | 00110110 (36 in hexadecimal) repeated 64 times (512 bits) |
| opad | = | 01011100 (5C in hexadecimal) repeated 64 times (512 bits) |

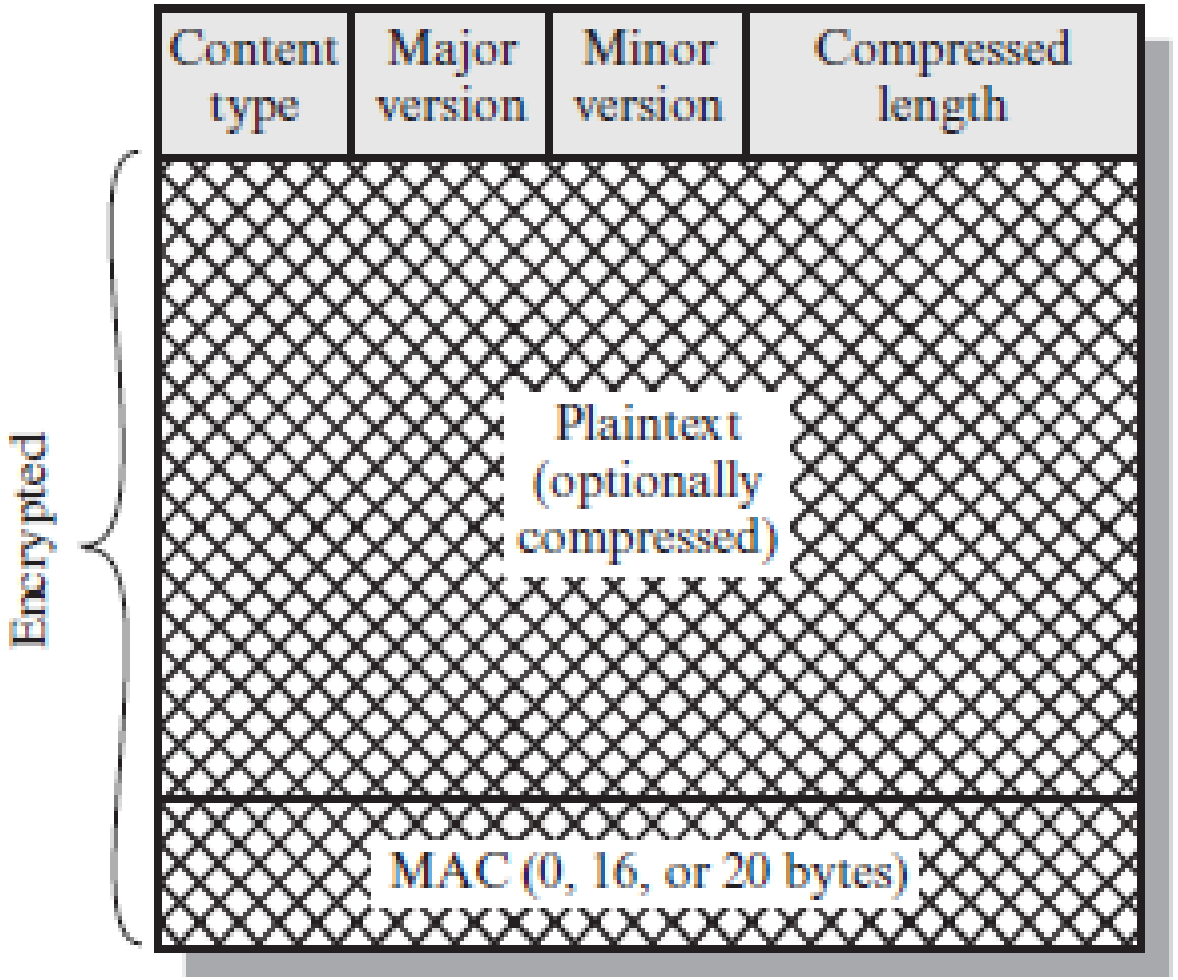



Figure 17.4 TLS Record Format

# Alert Protocol - TLS-related alerts to the peer entity

- Each message in this protocol consists of two bytes (Figure 17.5b). The first byte takes the value warning (1) or fatal (2) to convey the severity of the message. If the level is fatal, TLS immediately terminates the connection. Other connections on the same session may continue, but no new connections on this session may be established. The second byte contains a code that indicates the specific alert.

The following alerts are always fatal:

- **unexpected_message:** An inappropriate message was received.
- **bad_record_mac:** An incorrect MAC was received.
- **decompression_failure:** The decompression function received improper input (e.g., unable to decompress or decompress to greater than maximum allowable length).
- **handshake_failure:** Sender was unable to negotiate an acceptable set of security parameters given the options available.
- **illegal_parameter:** A field in a handshake message was out of range or inconsistent with other fields.

## Heartbeat Protocol

In the context of computer networks, a heartbeat is a periodic signal generated by hardware or software to indicate normal operation or to synchronize other parts of a system. A heartbeat protocol is typically used to monitor the availability of a protocol entity. In the specific case of TLS, a Heartbeat protocol was defined in 2012 in RFC 6250 (*Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) Heartbeat Extension*).

The Heartbeat protocol runs on top of the TLS Record Protocol and consists of two message types: `heartbeat_request` and `heartbeat_response`. The use of the Heartbeat protocol is established during Phase 1 of the Handshake protocol (Figure 17.6). Each peer indicates whether it supports heartbeats. If heartbeats are supported, the peer indicates whether it is willing to receive `heartbeat_request` messages and respond with `heartbeat_response` messages or only willing to send `heartbeat_request` messages.

# Attacks on TLS

*Attack Categories* We can group the attacks into four general categories:

- **Attacks on the handshake protocol:** As early as 1998, an approach to compromising the handshake protocol based on exploiting the formatting and implementation of the RSA encryption scheme was presented [BLEI98]. As countermeasures were implemented the attack was refined and adjusted to not only thwart the countermeasures but also speed up the attack [e.g., BARD12].
- **Attacks on the record and application data protocols:** A number of vulnerabilities have been discovered in these protocols, leading to patches to counter the new threats. As a recent example, in 2011, researchers Thai Duong and Juliano Rizzo demonstrated a proof of concept called BEAST (Browser Exploit Against SSL/TLS) that turned what had been considered only a theoretical vulnerability

into a practical attack [GOOD11]. BEAST leverages a type of cryptographic attack called a chosen-plaintext attack. The attacker mounts the attack by choosing a guess for the plaintext that is associated with a known ciphertext. The researchers developed a practical algorithm for launching successful attacks. Subsequent patches were able to thwart this attack. The authors of the BEAST attack are also the creators of the 2012 CRIME (Compression Ratio Info-leak Made Easy) attack, which can allow an attacker to recover the content of web cookies when data compression is used along with TLS [GOOD12]. When used to recover the content of secret authentication cookies, it allows an attacker to perform session hijacking on an authenticated web session.

- **Attacks on the PKI:** Checking the validity of X.509 certificates is an activity subject to a variety of attacks, both in the context of SSL/TLS and elsewhere. For example, [GEOR12] demonstrated that commonly used libraries for SSL/TLS suffer from vulnerable certificate validation implementations. The authors revealed weaknesses in the source code of OpenSSL, GnuTLS, JSSE, ApacheHttpClient, Weberknecht, cURL, PHP, Python and applications built upon or with these products.
- **Other attacks:** [MEYE13] lists a number of attacks that do not fit into any of the preceding categories. One example is an attack announced in 2011 by the German hacker group The Hackers Choice, which is a DoS attack [KUMA11]. The attack creates a heavy processing load on a server by overwhelming the target with SSL/TLS handshake requests. Boosting system load is done by establishing new connections or using renegotiation. Assuming that the majority of computation during a handshake is done by the server, the attack creates more system load on the server than on the source device, leading to a DoS. The server is forced to continuously recompute random numbers and keys.

## TLSv1.3

In 2014, the IETF TLS working group began work on a version 1.3 of TLS. The primary aim is to improve the security of TLS. As of this writing, TLSv1.3 is still in a draft stage, but the final standard is likely to be very close to the current draft. Among the significant changes from version 1.2 are the following:

- TLSv1.3 removes support for a number of options and functions. Removing code that implements functions no longer needed reduces the chances of potentially dangerous coding errors and reduces the attack surface. The deleted items include:
  - –Compression
  - –Ciphers that do not offer authenticated encryption
  - –Static RSA and DH key exchange
  - –32-bit timestamp as part of the Random parameter in the client_hello message

–Renegotiation
–Change Cipher Spec Protocol
–RC4
–Use of MD5 and SHA-224 hashes with signatures

- TLSv1.3 uses Diffie–Hellman or Elliptic Curve Diffie–Hellman for key exchange and does not permit RSA. The danger with RSA is that if the private key is compromised, all handshakes using these cipher suites will be compromised. With DH or ECDH, a new key is negotiated for each handshake.
- TLSv1.3 allows for a "1 round trip time" handshake by changing the order of message sent with establishing a secure connection. The client sends a Client Key Exchange message containing its cryptographic parameters for key establishment before a cipher suite has been negotiated. This enables a server to calculate keys for encryption and authentication before sending its first response. Reducing the number of packets sent during this handshake phase speeds up the process and reduces the attack surface.

These changes should improve the efficiency and security of TLS.

## 17.3 HTTPS

HTTPS (HTTP over SSL) refers to the combination of HTTP and SSL to implement secure communication between a Web browser and a Web server. The HTTPS capability is built into all modern Web browsers. Its use depends on the Web server supporting HTTPS communication. For example, some search engines do not support HTTPS.

The principal difference seen by a user of a Web browser is that URL (uniform resource locator) addresses begin with https:// rather than http://. A normal HTTP connection uses port 80. If HTTPS is specified, port 443 is used, which invokes SSL.

When HTTPS is used, the following elements of the communication are encrypted:

- URL of the requested document
- Contents of the document
- Contents of browser forms (filled in by browser user)
- Cookies sent from browser to server and from server to browser
- Contents of HTTP header

HTTPS is documented in RFC 2818, *HTTP Over TLS*. There is no fundamental change in using HTTP over either SSL or TLS, and both implementations are referred to as HTTPS.

### Connection Initiation

For HTTPS, the agent acting as the HTTP client also acts as the TLS client. The client initiates a connection to the server on the appropriate port and then sends the TLS ClientHello to begin the TLS handshake. When the TLS handshake has

There are three levels of awareness of a connection in HTTPS. At the HTTP level, an HTTP client requests a connection to an HTTP server by sending a connection request to the next lowest layer. Typically, the next lowest layer is TCP, but it also may be TLS/SSL. At the level of TLS, a session is established between a TLS client and a TLS server. This session can support one or more connections at any time. As we have seen, a TLS request to establish a connection begins with the establishment of a TCP connection between the TCP entity on the client side and the TCP entity on the server side.

## Connection Closure

An HTTP client or server can indicate the closing of a connection by including the following line in an HTTP record: `Connection: close`. This indicates that the connection will be closed after this record is delivered.

The closure of an HTTPS connection requires that TLS close the connection with the peer TLS entity on the remote side, which will involve closing the underlying TCP connection. At the TLS level, the proper way to close a connection is for each side to use the TLS alert protocol to send a `close_notify` alert. TLS implementations must initiate an exchange of closure alerts before closing a connection. A TLS implementation may, after sending a closure alert, close the connection without waiting for the peer to send its closure alert, generating an "incomplete close". Note that an implementation that does this may choose to reuse the session. This should only be done when the application knows (typically through detecting HTTP message boundaries) that it has received all the message data that it cares about.

HTTP clients also must be able to cope with a situation in which the underlying TCP connection is terminated without a prior `close_notify` alert and without a `Connection: close` indicator. Such a situation could be due to a programming error on the server or a communication error that causes the TCP connection to drop. However, the unannounced TCP closure could be evidence of some sort of attack. So